Database Systems

Lecture 4: Relational Model

Assistant Professor
Yumeng Song
Department of Computer Science
Aalborg University
yumengs@cs.aau.dk
Fall 2025

Learning Objectives

- After completing this lecture, you will be able to:
  - Explain the relational model
  - Create non-trivial relational algebra queries
  - Use the various join types in relational algebra queries

Agenda

- Steps of Database Design
- The Relational Model
  - Structure of relational databases
  - Database schema
- Relational Algebra
  - Select
  - Project
  - Cartesian product
  - Join (derived, a combination of select and Cartesian product)
  - Union
  - Difference
  - Intersection (derived, a combination of multiple differences)
  - Rename
  - Group (derived, partitions tuples and applies aggregation)

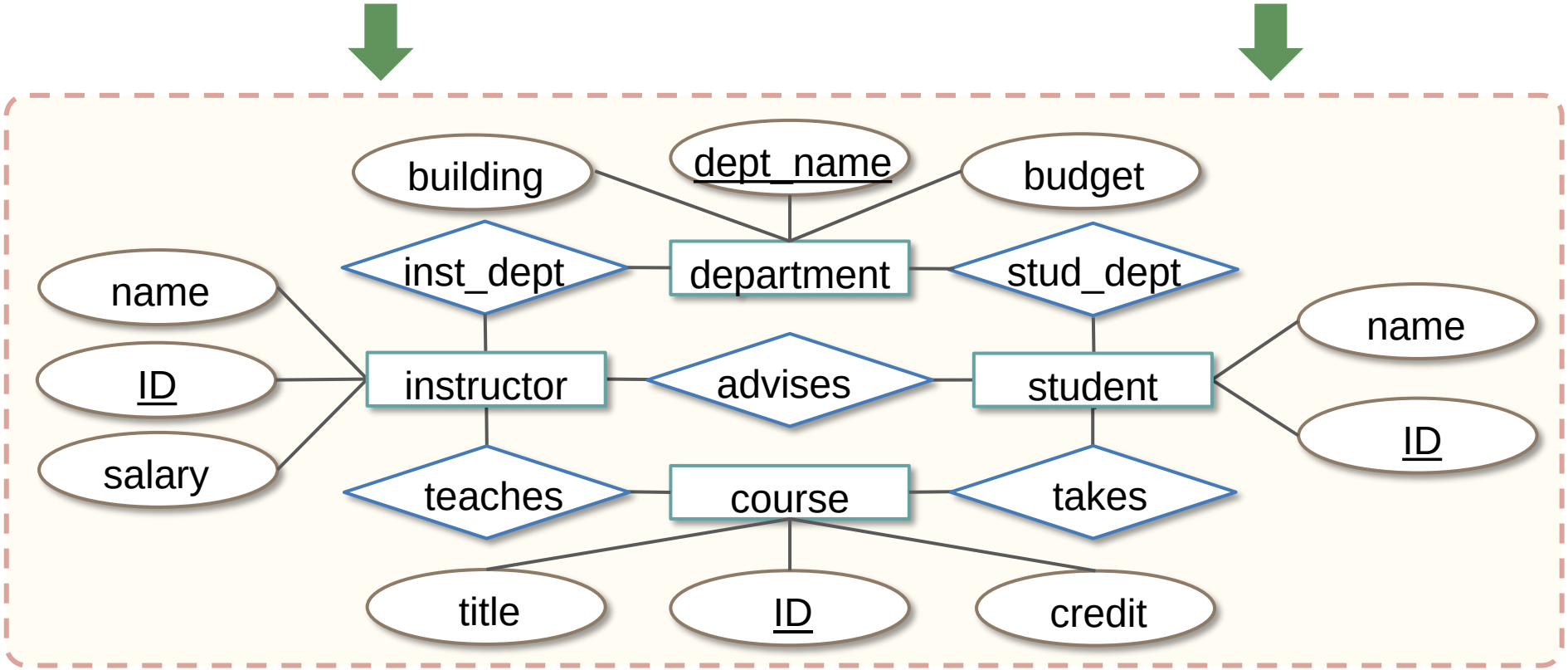
Example Design – University Database

- Step 1: Requirement collection and analysis (What are we dealing with?)

- “Instructors are in different departments”
- “Instructors teach courses”
- “Students take courses”
- …

Example Design – University Database

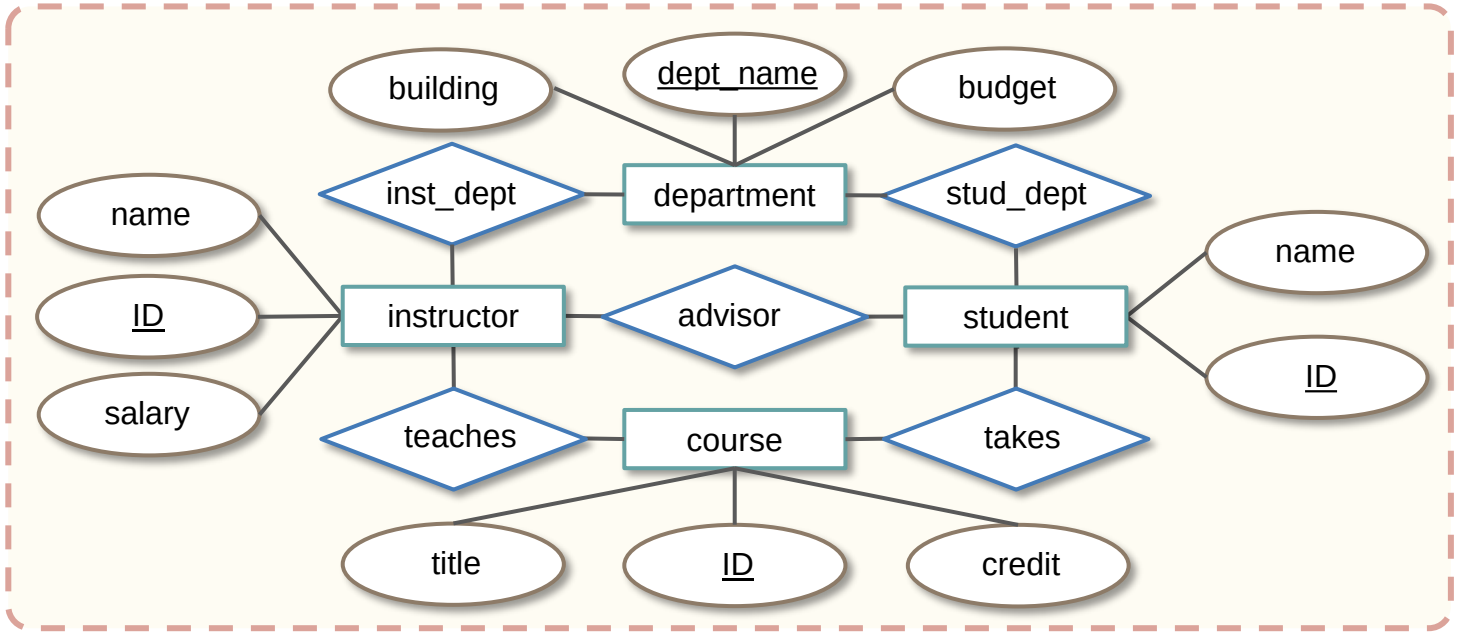
- Step 2: Conceptual design (What data and relationships have to be captured?)
  - Requirements
    - “Instructors are in different departments”, “Instructors teach courses”, “Students take courses”, …



Easy for people to understand!

Example Design – University Database

- Step 3: Logical design (How to structure data in a specific model?)



- *instructor (ID, name, dept_name, salary)*
- *student (ID, name, dept_name, tot_cred)*
- *course (course_id, title, dept_name, credits)*
- *department (dept_name, building, budget)*

Easy for computer to understand!

Example Design – University Database

- Step 4: Physical Design (Which adaptations and optimizations does a specific DBMS require?)
  - *instructor (ID, name, dept_name, salary)*
  - *student (ID, name, dept_name, tot_cred)*
  - *course (course_id, title, dept_name, credits)*
  - *department (dept_name, building, budget)*

Table in DB

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

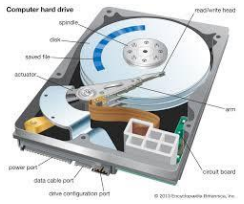
Example Design – University Database

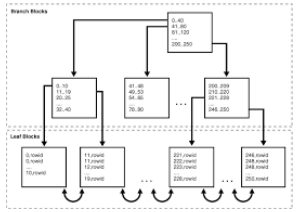
- Step 4: Physical Design (Which adaptations and optimizations does a specific DBMS require?)

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Memory, pages, data structures, indexes, devices, …




Steps of Database Design

Real World

01 — Requirement Collection and Analysis

Data Requirement

02 — Conceptual Design

Conceptual Schema

03 — Logical Design

We are here!

Logical Schema

04 — Physical Design

Physical Database

Juan's part

Consult database users to understand data requirements

Develop ER diagram with entities and relationship

Covert ER diagram to logical database schema

Covert logical database schema to physical database

Agenda

- Steps of Database Design
- The Relational Model
  - Structure of relational databases
  - Database schema
- Relational Algebra
  - Select
  - Project
  - Cartesian product
  - Join (derived, a combination of select and Cartesian product)
  - Union
  - Difference
  - Intersection (derived, a combination of multiple differences)
  - Rename
  - Group (derived, partitions tuples and applies aggregation)

Relations (Tables)

- Domain: a set of values of the same data type.
  - Example: D_1={BIO-101, BIO-301}, D_2={BIO, CS, EE}, D_3={4,3}

- Cartesian Product
  - Given a set of domains $D_1, D_2, \ldots, D_n$, their Cartesian product is

$$D_1 \times D_2 \times \cdots \times D_n = \{(d_1, d_2, \ldots, d_n) | d_i \in D_i, \mathrm{i} = 1,2, \ldots, \mathrm{n}\}$$

  - Example: D_1={BIO-101, BIO-301}, D_2={BIO, CS, EE}, D_3={4,3}

  - $D_1 \times D_2 \times D_3$ ={ (BIO-101, BIO, 4), (BIO-101, BIO, 3), (BIO-101, CS, 4), (BIO-101, CS, 3), (BIO-101, EE, 4), (BIO-101, EE, 3), (BIO-301,BIO, 4), (BIO-301,BIO, 3), (BIO-301,CS, 4), (BIO-301,CS, 3), (BIO-301,EE, 4), (BIO-301,EE, 3) }

Relations (Tables)

- Relation
  - Assume $D_1, D_2, \ldots, D_n$ are domains, relation R is a set of elements from the Cartesian product, i.e., $R \subseteq D_1 \times D_2 \times \cdots \times D_n$.

  - Example:
    - D_1={BIO-101, BIO-301}, D_2={BIO, CS, EE}, D_3={4,3}
    - $D_1 \times D_2 \times D_3$ ={ (BIO-101, BIO, 4), (BIO-101, BIO, 3), (BIO-101, CS, 4), (BIO-101, CS, 3), (BIO-101, EE, 4), (BIO-101, EE, 3), (BIO-301, BIO, 4), (BIO-301, BIO, 3), (BIO-301, CS, 4), (BIO-101, CS, 3), (BIO-301, EE, 4), (BIO-301, EE, 3) }

    - Relation examples:
    - R_1={(BIO-101, BIO,4), (BIO-301, BIO,4)}
    - R_2={(BIO-101, BIO,3), (BIO-301, BIO,3)}
    - R_3={(BIO-101, BIO,3), (BIO-301, BIO,3), (BIO-101, CS, 3)}
    - …
    - ✓ *Any subset can be a relation.*

Relations (Tables)

- A relational database consists of a collection of **tables**.
  - A row in a table represents a *relationship* among a set of values.
  - Since a table is a collection of such relationships, there is a close correspondence between the concept of *table* and the mathematical concept of *relation*.
- In the relational model the term **relation** is used to refer to a table.

course

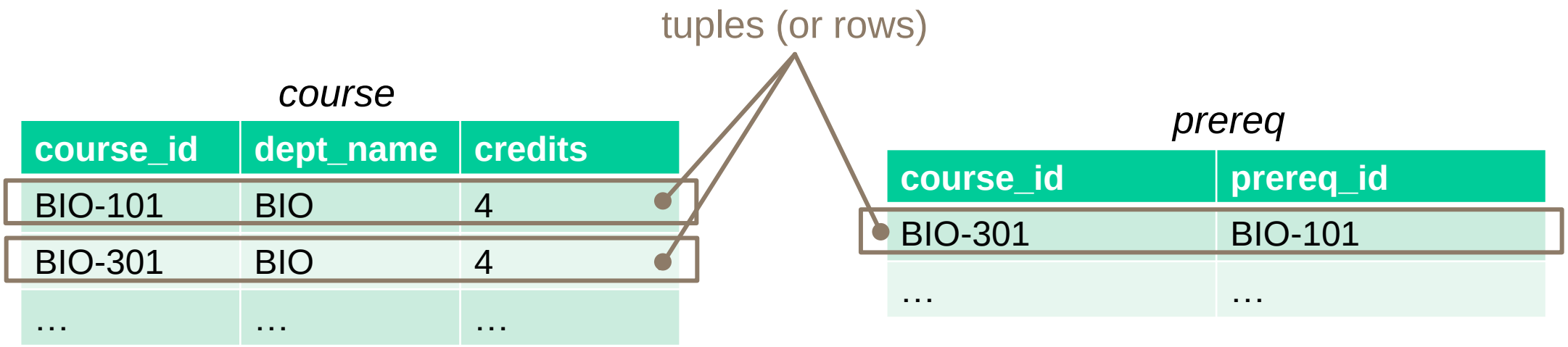
course_id	dept_name	credits
BIO-101	BIO	4
BIO-301	BIO	4
…	…	…

prereq

course_id	prereq_id
BIO-301	BIO-101
…	…

Tuples

- The term **tuple** is used to refer to a row.



Tuples

- The tuples in a relation are unordered
  - Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
  - Example: instructor relation with unordered tuples

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
983456	Kim	Elec. Eng.	80000

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

These relations have the same information content.

Tuples

- The tuples in a relation are unique
  - No duplicate tuples are allowed (each tuple appears at most once)
  - Example: instructor relation with unique tuples

ID	name	dept_name	salary
45565	Katz	Comp. Sci.	75000
33456	Gold	Physics	87000
33456	Gold	Physics	87000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
83821	Brandt	Comp. Sci.	92000

✗ **Wrong!**

ID	name	dept_name	salary
45565	Katz	Comp. Sci.	75000
33456	Gold	Physics	87000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000

O Correct version

Attributes

- The term **attribute** refers to a column of a table.
  - The set of allowed values for each attribute is called the domain of the attribute
  - The special value null is a member of every domain. Indicated that the value is “unknown”.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
983456	Kim	Elec. Eng.	80000

instructor

- Example:
  - Attribute: ID, name, dept_name, salary
  - The domain of the dept_name attribute is the set of all possible department names.

Keys

- Superkey
  - A superkey is a set of one or more attributes of a relation such that the values of these attributes are sufficient to uniquely identify each tuple in the relation.
  - If two tuples t_1 and t_2 in a relation are different, then their values on the superkey attributes must also be different.
  - *This property must hold for all possible valid instances of the relation, not just for the current data stored in the table.*
  - Example: instructor = (ID, name, dept_name, salary)
    - Which of the following attribute sets are superkeys?
      - {ID}
      - {name}
      - {dept_name}
      - {ID, name}
      - {ID, name, dept_name}
      - {dept_name, salary}
      - {ID, name, dept_name, salary}

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	75000
23220	Einstein	Physics	95000
…	…	…	…

instructor

Keys

- Candidate key
  - Superkey K is a candidate key if K is minimal
    - A superkey may contain extraneous attributes.
    - We are often interested in superkeys for which no proper subset is a superkey.
  - Example: instructor = (ID, name, dept_name, salary)
    - Superkey: {ID}, {ID, name}, {ID, name, dept_name}, {ID, name, dept_name, salary}
    - Candidate key: {ID}
    - {ID, name}, {ID, name, dept_name} are not candidate keys
    - Suppose that a combination of name and dept_name is sufficient to distinguish among members of the instructor relation
    - Both {ID} and {name, dept_name} are candidate keys

Keys

- Primary key
  - One of the candidate keys is selected to be the primary key.
- Which one?
  - Primary keys must be chosen with care.
  - The primary key should be chosen such that its attribute values are never, or are very rarely, changed.
- Examples
  - *instructor (ID, name, dept_name, salary)*
  - *course (course_id, title, dept_name ,credits)*
  - *department (dept_name, building, budget)*
  - *student (ID, name, dept_name, tot_cred)*
  - *classroom (building, room_number, capacity)*

Keys

- Foreign key
  - A foreign key is an attribute that refers to the candidate key of another table.
  - Value in one relation must appear in another

- Examples
  - *instructor (ID, name, dept_name, salary)*
  - *department (dept_name, building, budget)*

  - *dept_name in instructor is a foreign key from instructor referencing department*
  - *Referencing relation: instructor*
  - *Referenced relation: department*

Agenda

- Steps of Database Design
- The Relational Model
  - Structure of relational databases
  - Database schema
- Relational Algebra
  - Select
  - Project
  - Cartesian product
  - Join (derived, a combination of select and Cartesian product)
  - Union
  - Difference
  - Intersection (derived, a combination of multiple differences)
  - Rename
  - Group (derived, partitions tuples and applies aggregation)

Database Schema

- Relation schema: the logical structure of the relation.
  - Given attributes $A_1, A_2, \ldots, A_n$, $R = (A_1, A_2, \ldots, A_n)$ is a relation schema.
  - Example:
    - instructor = (ID, name, dept_name, salary)

- Relation instance: a snapshot of the data in the relation at a given instant in time
  - A relation instance r defined over R is denoted by $r(R)$.
  - An element t of relation r is called a tuple and is represented by a row in a table.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
983456	Kim	Elec. Eng.	80000

instructor

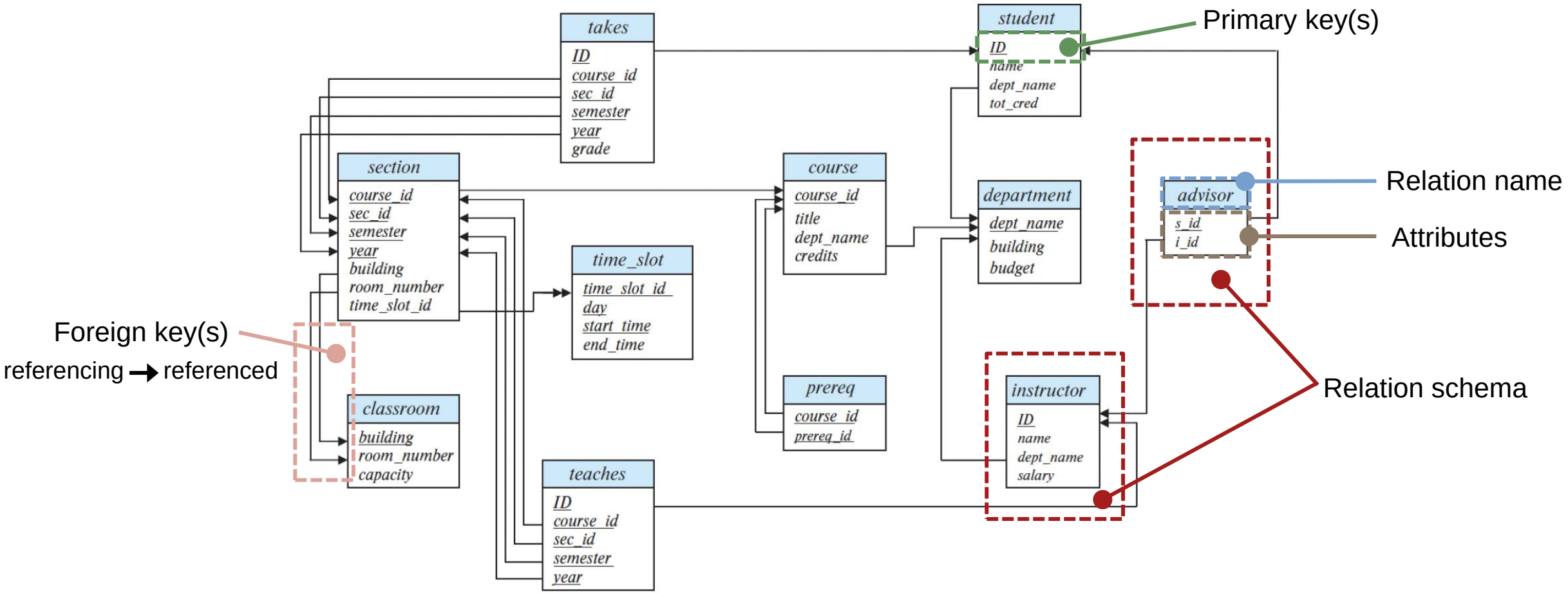
Relation instance

Database Schema

- Database schema
  - The set of all relation schemas in the database
  - Example: University Database
    - instructor (ID, name, dept_name, salary)
    - course (course_id, title, dept_name ,credits)
    - department (dept_name, building, budget)
    - section (course_id, sec_id, semester, year, building, room_number, time_slot_id)
    - teaches (ID, course_id, sec_id, semester, year)
    - student (ID, name, dept_name, tot_cred)
    - advisor (s_id, i_id)
    - takes (ID, course_id, sec_id, semester, year, grade)
    - classroom (building, room_number, capacity)
    - time slot (time_slot_id, day, start_time, end_time)
- Database instance
  - The actual contents of the database at a given moment.

Schema Diagram

- A database schema, along with primary key and foreign-key constraints, can be depicted by schema diagrams.



Agenda

- Steps of Database Design
- The Relational Model
  - Structure of relational databases
  - Database schema
- Relational Algebra
  - Select
  - Project
  - Cartesian product
  - Join (derived, a combination of select and Cartesian product)
  - Union
  - Difference
  - Intersection (derived, a combination of multiple differences)
  - Rename
  - Group (derived, partitions tuples and applies aggregation)

Relational algebra

- A mathematical way to formulate queries on relations (i.e., tables).
  - Input: one or multiple relations
  - Output: a new relation

- Fundamental operations
  - Selection σ
  - Projection π
  - Rename ρ
  - Cartesian product $\times$
  - Union $\cup$
  - Difference $-$

Unary operations
Operate on one relation

Binary operations
Operate on pairs of relations

- ✓ Any relational algebra query can be expressed with the set of fundamental operations only.
- ✓ Removing any one of these operations reduces the expressive power.
- ✓ Operations can be combined (with some rules)

Relational algebra

- Relational algebra vs. SQL
  - Both are query languages for relations
  - SQL: used by the user -- Relational algebra: core logic inside the database
  - One SQL query may be executed using different relational algebra expressions
  - Example: Find all instructors in the CS department with salary > 80,000
    - Filter by department first? Or filter by salary first?
    - Same result, but different performance
  - Database chooses the best execution plan (relational algebra) even though we write SQL

Select Operation σ

- The select operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
  - Selection predicate *p* consists of
    - Logic operators: ∨(or), ∧(and), ¬(not)
    - Arithmetic comparison operators: <, ≤, =, >, ≥, ≠
    - Attribute names of the argument relations or constants as operands.
- Example:
  - Select those tuples of the instructor relation where the instructor is in the "Physics" department.
  - $p: dept\_name = "Physics"$
  - $r: instructor$
  - $\sigma_{dept\_name="Physics"}(instructor)$

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
…	…	…	…

instructor

Select Operation σ

- Example:
  - Select those tuples of the instructor relation where the instructor is in the "Physics" department.
    - $\sigma_{dept\_name="Physics"}(instructor)$

ID	name	dept_name	salary
22222	**Einstein**	**Physics**	**95000**
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	**Gold**	**Physics**	**87000**
76543	Singh	Finance	80000

instructor

Result

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select Operation σ

- We allow comparisons using $<, \leq, =, >, \geq, \neq$ in the selection predicate.
- Example:
  - Select the instructors whose salary is greater than 80000
  - $\sigma_{salary>80000}(instructor)$

ID	name	dept_name	salary
22222	**Einstein**	**Physics**	**95000**
12121	**Wu**	**Finance**	**90000**
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	**Brandt**	**Comp. Sci.**	**92000**
15151	Mozart	Music	40000
33456	**Gold**	**Physics**	**87000**
76543	Singh	Finance	80000

instructor

Result

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
83821	Brandt	Comp. Sci.	92000
33456	Gold	Physics	87000

Select Operation σ

- We can combine several predicates into a larger predicate by using the connectives: ∨(or), ∧(and), ¬(not)
- Example:
  - Select the Physics instructors whose salary is greater than 90000
  - $\sigma_{dept\_name=Physics \wedge salary>90000}(instructor)$

ID	name	dept_name	salary
22222	**Einstein**	**Physics**	**95000**
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	**Gold**	**Physics**	**87000**
76543	Singh	Finance	80000

instructor

Result

ID	name	dept_name	salary
22222	Einstein	Physics	95000

Project Operation π

- A unary operation that returns its argument relation, with certain attributes left out.
- Notation: $\pi_{A_1,A_2,A_3,\ldots,A_k}(r)$
  - $A_1, A_2, A_3, \ldots, A_k$ are attribute names and r is a relation.
  - The result is a relation of k columns obtained by removing the columns that are not specified.
  - Duplicate rows removed from result, since relations are sets.

- Example:
  - Return the dept_name attribute of the instructor.
  - $\pi_{dept\_name}(instructor)$

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
…	…	…	…

instructor

Project Operation π

- Example:
  - Return the dept_name attribute of the instructor.
  - $\pi_{dept_{name}}(instructor)$

ID	name	dept_name	salary
22222	Einstein	**Physics**	95000
12121	Wu	**Finance**	90000
32343	El Said	**History**	60000
45565	Katz	**Comp. Sci.**	75000
983456	Kim	**Elec. Eng.**	80000
76766	Crick	**Biology**	72000
10101	Srinivasan	**Comp. Sci.**	65000
58583	Califieri	**History**	62000
83821	Brandt	**Comp. Sci.**	92000
15151	Mozart	**Music**	40000
33456	Gold	**Physics**	87000
76543	Singh	**Finance**	80000

instructor

dept_name
Physics
Finance
History
Comp. Sci.
Elec. Eng.
Biology
Comp. Sci.
History
Comp. Sci.
Music
Physics
Finance

Remove duplicate rows

Result

dept_name
Physics
Finance
History
Comp. Sci.
Elec. Eng.
Biology
Music

Composition of Relational Operations

- Consider the query -- Find the names of all instructors in the Physics department.

➢ $\pi_{name}(\sigma_{dept\_name="Physics"}(instructor))$

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

instructor

$\sigma_{dept\_name="Physics"}(instructor)$

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

$\pi_{name}(\sigma_{dept\_name="Physics"}(instructor))$

name
Einstein
Gold

Result

Composition of Relational Operations

- Consider the query -- Find the names of all instructors in the Physics department.
- Think about this: $\sigma_{dept\_name="Physics"}(\pi_{name}(instructor))$

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

instructor

$\pi_{name}(instructor)$

name
Einstein
Wu
El Said
Katz
Kim
Crick
Srinivasan
Califieri
Brandt
Mozart
Gold
Singh

$\sigma_{dept\_name="Physics"}(\pi_{name}(instructor))$

Wrong!

- The projection goes before the selection here
- Since the projection eliminates "*dept_name*', the selection cannot be performed

Exercise 1

- Query -- Find the IDs of all instructors whose salary is less than 82,000.
  - *Q1. Write the Relational Algebra expression for the query.*
  - *Q2. Based on the following table, give the query result.*

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

instructor

Exercise 1 -- Solution

- Query -- Find the IDs of all instructors whose salary is less than 82000.
  - *Q1. Write the Relational Algebra expression for the query.*
  - *Q2. Based on the following table, give the query result.*

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	**El Said**	**History**	**60000**
45565	**Katz**	**Comp. Sci.**	**75000**
983456	**Kim**	**Elec. Eng.**	**80000**
76766	**Crick**	**Biology**	**72000**
10101	**Srinivasan**	**Comp. Sci.**	**65000**
58583	**Califieri**	**History**	**62000**
83821	Brandt	Comp. Sci.	92000
15151	**Mozart**	**Music**	**40000**
33456	Gold	Physics	87000
76543	**Singh**	**Finance**	**80000**

instructor

- *A1.* $\pi_{ID}\left(\sigma_{salary<82000}(instructor)\right)$
- *A2.* Result:

ID
32343
45565
983456
76766
10101
58583
15151
76543

Cartesian-Product Operation ×

- The Cartesian-product operation (denoted by ×) allows us to combine information from any two relations.
- Example
  - The Cartesian product of the relations *instructor* and *teaches* is written as:

$$instructor \times teaches$$

  - *instructor* (ID, name, dept_name, salary)
  - *teaches* (ID, course_id, sec_id, semester, year)

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

instructor

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

teaches

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...

Result

Cartesian-Product Operation ×

- Most of the resulting rows in $instructor \times teaches$ have information about instructors who did NOT teach a particular course.
- To get only those tuples of "$instructor \times teaches$" that pertain to instructors and the courses that they taught, we write:

$$\sigma_{instructor.id=teaches.id}(instructor \times teaches)$$

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

instructor

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

teaches

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Result

Cartesian-Product Operation ×

- $\sigma_{instructor.id=teaches.id}(instructor \times teaches)$

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

$instructor \times teaches$

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Result

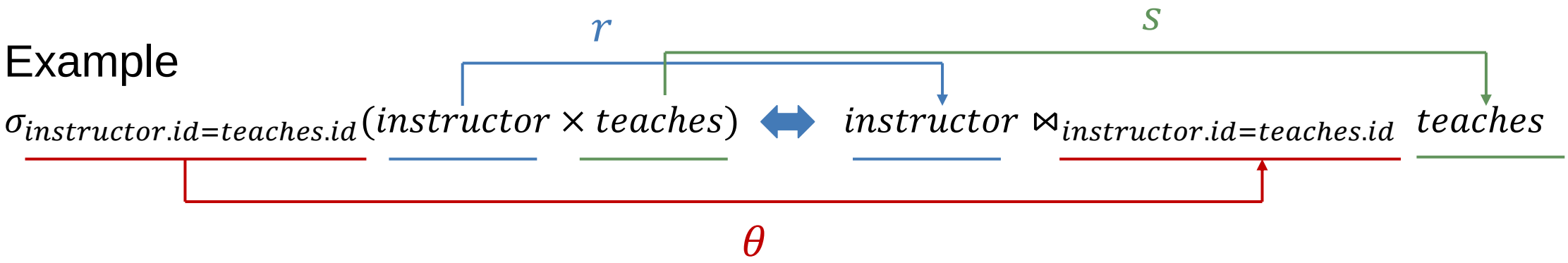
Theta Join Operation $\bowtie_\theta$

- The **join** operation combine a select operation and a Cartesian-Product operation into a single operation.

- A **theta join $\bowtie_\theta$** is a join operation with a general condition.
- Given two relations and **θ(theta)**
  - $r(R)$ and $s(S)$
  - θ is an arbitrary predicate over the tables' attributes

$$r \bowtie_\theta s = \sigma_\theta(r \times s)$$

- Example



Theta Join Operation $\bowtie_\theta$

- Example
  - Find the students whose score higher in quiz2 than quiz1
  - $quiz1 \bowtie_{quiz1.name=quiz2.name \wedge quiz2.score>quiz1.score} quiz2$

join two scores of same student

select the rows that score higher in quiz2

$quiz1$

name	score
Anna	80
Bob	70
Cathy	90
David	80

$quiz2$

name	score
Anna	90
Bob	60
Cathy	100
Ming	100

Result

quiz1.name	quiz2.name	quiz1.score	quiz2.score
Anna	Anna	80	90
Cathy	Cathy	90	100

Theta Join Operation $\bowtie_\theta$

- Example
  - For those students who have made donations, find their donations.

$student$

name	dept_name
Anna	Comp. Sci.
Bob	Physics
Cathy	Music
David	Physics

Result

student.name	dept_name	donation.name	amount
Anna	Comp. Sci.	Anna	150
David	Physics	David	180

$$student \bowtie_{student.name=donation.name} donation$$

$donation$

name	amount
Alice	100
Anna	150
Chris	120
David	180

Result

student.name	dept_name	amount
Anna	Comp. Sci.	150
David	Physics	180

$$\pi_{student.name,dept\_name,amount}(student \bowtie_{student.name=donation.name} donation)$$

Natural Join Operation ⋈

- Natural Join ⋈ : a special case of theta join
  - The join is performed based on the common attributes of the two relations
  - Each common attribute appears only once in the result

- Example

$$student \bowtie donation$$

$student$

name	dept_name
Anna	Comp. Sci.
Bob	Physics
Cathy	Music
David	Physics

$donation$

name	amount
Alice	100
Anna	150
Chris	120
David	180

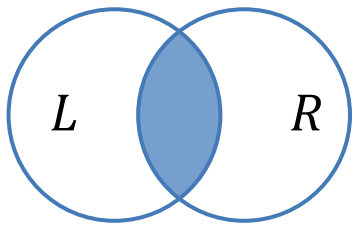
Result

student.name	dept_name	amount
Anna	Comp. Sci.	150
David	Physics	180

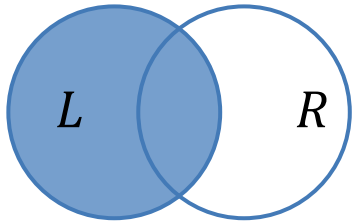
Outer joins

- How do we treat unmatched tuples, i.e., tuples without matching join partners in the other relation (dangling tuples)?

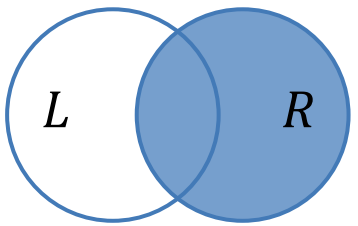
- ⟕ Left join
  - keep dangling tuples in the left operand relation
- ⟖ Right join
  - keep dangling tuples in the right operand relation
- ⟗ (Full) outer Join
  - keep dangling tuples of both operand relations



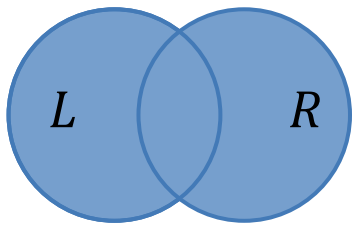
$L \bowtie R$



$L \bowtie R$



$L \bowtie R$



$L \bowtie R$

Outer joins

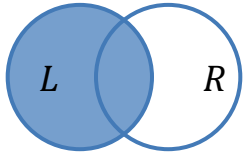
- Example
  - For each student, find the amount of his/her donation
    - $student \bowtie donation$
  - For each donator, find the department he/she is in
    - $student \bowtie donation$
  - For each student and each donator, show all students and all donators, even if some have no matching records
    - $student \bowtie donation$

$student$ (L)

name	dept_name
Anna	Comp. Sci.
Bob	Physics
Cathy	Music
David	Physics

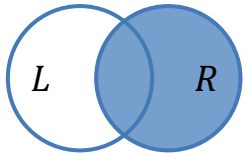
$donation$ (R)

name	amount
Alice	100
Anna	150
Chris	120
David	180



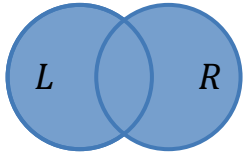
$student \bowtie donation$

name	dept_name	amount
Anna	Comp. Sci.	150
Bob	Physics	**null**
Cathy	Music	**null**
David	Physics	180



$student \bowtie donation$

name	dept_name	amount
Alice	**null**	100
Anna	Comp. Sci.	150
Chris	**null**	120
David	Physics	180



$student \bowtie donation$

name	dept_name	amount
Alice	**null**	100
Anna	Comp. Sci.	150
Bob	Physics	**null**
Cathy	Music	**null**
Chris	**null**	120
David	Physics	180

Exercise 2

- Given two relations $student$ and $course$, compute:
  - *Q1. Cartesian-product operation:* $student \times course$
  - *Q2. Theta join operation:* θ: $student.stu\_ID = course.stu\_ID$
  - *Q3. Natural join operation:* $student \bowtie course$

$student$

stu_ID	stu_name	age
1	John	20
2	Sarah	19
3	Mike	22

$course$

cou_ID	cou_name	stu_ID
101	Math	1
102	History	3
103	Physics	2

Exercise 2 - Solution

- Given two relations $student$ and $course$, compute:
  - *Q1. Cartesian-product operation:* $student \times course$

$student$

stu_ID	stu_name	age
1	John	20
2	Sarah	19
3	Mike	22

$course$

cou_ID	cou_name	stu_ID
101	Math	1
102	History	3
103	Physics	2

  - *A1. Result of* $student \times course$

stu_ID	stu_name	age	cou_ID	cou_name	stu_ID
1	John	20	101	Math	1
1	John	20	102	History	3
1	John	20	103	Physics	2
2	Sarah	19	101	Math	1
2	Sarah	19	102	History	3
2	Sarah	19	103	Physics	2
3	Mike	22	101	Math	1
3	Mike	22	102	History	3
3	Mike	22	103	Physics	2

Exercise 2 -- Solution

- Given two relations $student$ and $course$, compute:
  - *Q2. Theta join operation: θ: $student.stu\_ID = course.stu\_ID$*

$student$

stu_ID	stu_name	age
1	John	20
2	Sarah	19
3	Mike	22

$course$

cou_ID	cou_name	stu_ID
101	Math	1
102	History	3
103	Physics	2

  - *A2. Result of $student \bowtie_{student.stu\_ID=course.stu\_ID} course$*

stu_ID	stu_name	age	cou_ID	cou_name	stu_ID
1	John	20	101	Math	1
2	Sarah	19	103	Physics	2
3	Mike	22	102	History	3

Exercise 2 -- Solution

- Given two relations $student$ and $course$, compute:
  - *Q3. Natural join operation:* $student \bowtie course$

$student$

stu_ID	stu_name	age
1	John	20
2	Sarah	19
3	Mike	22

$course$

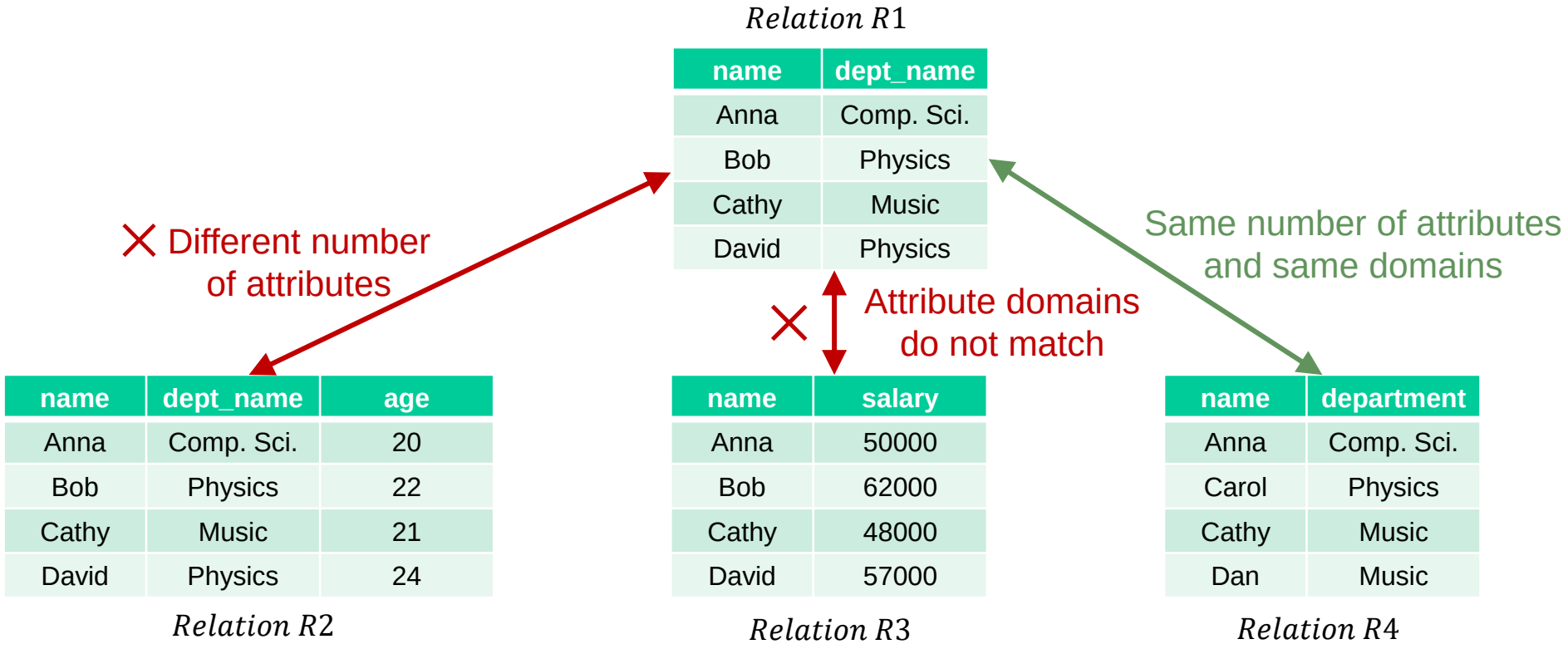
cou_ID	cou_name	stu_ID
101	Math	1
102	History	3
103	Physics	2

  - *A3. Result of* $student \bowtie course$

stu_ID	stu_name	age	cou_ID	cou_name
1	John	20	101	Math
2	Sarah	19	103	Physics
3	Mike	22	102	History

Set operations

- The set operations union, intersection, and difference can also be applied to relations.
- Requirement: both involved relations must be union-compatible
  - They have the same number of attributes
  - The domain of each attribute in column order is the same in both relations



Union Operation ∪

- The union operation combines two relations, and the result contains all tuples from both relations without duplicates.
- Notation: $r \cup s$
  - r and s are two relations
- Example
  - Find the ids of all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

section

$$\pi_{course\_id}\left(\sigma_{semester="Fall" \wedge year=2017}(section)\right)$$

∪

$$\pi_{course\_id}\left(\sigma_{semester="Spring" \wedge year=2018}(section)\right)$$

Result:

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Union Operation ∪

- Example
  - Find the persons who are either students or volunteers
  - $student \cup volunteer$

$student$

name	age
Anna	19
Bob	22
Cathy	20
David	21

∪

$volunteer$

name
Alice
Anna
Chris
David

✗ Wrong!

  - $\pi_{\text{name}}(student) \cup volunteer$

$\pi_{\text{name}}(student)$

name
Anna
Bob
Cathy
David

∪

$volunteer$

name
Alice
Anna
Chris
David

=

$Result$

name
Alice
Anna
Bob
Cathy
Chris
David

O Correct version

Difference Operation –

- The difference operation removes all tuple from the first relation that are also contained in the second relation.
- Notation: $r - s$ or $r \backslash s$
  - r and s are two relations
- Example
  - Find the ids of all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

section

$$\pi_{course\_id}\left(\sigma_{semester="Fall" \wedge year=2017}(section)\right)$$

–

$$\pi_{course\_id}\left(\sigma_{semester="Spring" \wedge year=2018}(section)\right)$$

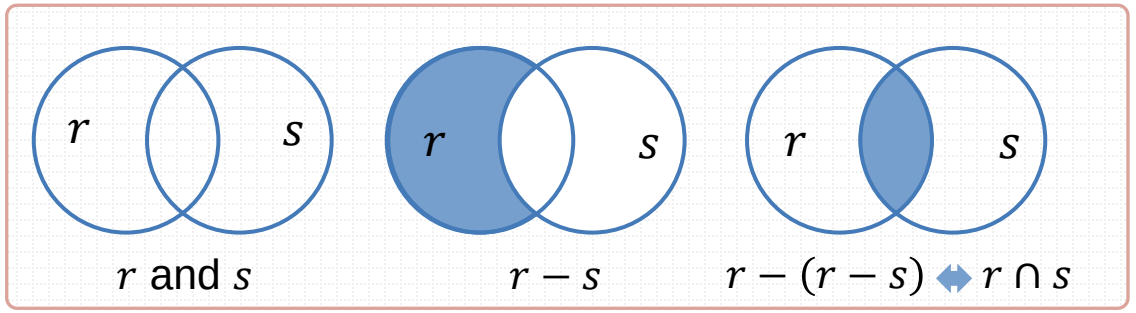
The difference keeps the tuples in the blue set and removes those that also appear in the green set.

Result:

course_id
CS-347
PHY-101

Intersection Operation ∩

- The intersection operation finds tuples that are in both the input relations.
- Notation: $r \cap s$
  - r and s are two relations
- Intersection is not a fundamental operation!
  - $r \cap s$ can be expressed as difference $r - (r - s)$



- Example
  - Find the ids of all courses taught in both the Fall 2017 and the Spring 2018 semesters

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

section

$$\pi_{course\_id}\left(\sigma_{semester="Fall" \wedge year=2017}(section)\right)$$

$$\cap$$

$$\pi_{course\_id}\left(\sigma_{semester="Spring" \wedge year=2018}(section)\right)$$

The intersection keeps only the tuples that appear in both the blue set and the green set.

Result:

course_id
CS-101

Exercise 3

- Given two relations $course\_A$ and $course\_B$, compute
  - *Q1. Union operation:* $course\_A \cup course\_B$
  - *Q2. Intersection operation:* $course\_A \cap course\_B$
  - *Q3. Difference operation (in course_A, not in course_B):* $course\_A - course\_B$

$course\_A$

cou_ID	cou_name
101	Math
102	History
103	Physics

$course\_B$

cou_ID	cou_name
102	History
103	Physics
104	Chemistry

Exercise 3 -- Solution

- Given two relations $course\_A$ and $course\_B$, compute
  - *Q1. Union operation:* $course\_A \cup course\_B$
  - *A1.* $course\_A \cup course\_B$

cou_ID	cou_name
101	Math
102	History
103	Physics
104	Chemistry

  - *Q2. Intersection operation:* $course\_A \cap course\_B$
  - *A2.* $course\_A \cap course\_B$

cou_ID	cou_name
102	History
103	Physics

  - *Q3. Difference operation (in* $course\_A$*, not in* $course\_B$*):* $course\_A - course\_B$
  - *A3.* $course\_A - course\_B$

cou_ID	cou_name
101	Math

$course\_A$

cou_ID	cou_name
101	Math
102	History
103	Physics

$course\_B$

cou_ID	cou_name
102	History
103	Physics
104	Chemistry

Rename Operation ρ

- The rename operation ρ assigns a new name to a relation or its attributes
- $\rho_x(E)$
  - Renames the result of expression E under the name x
  - Example: $\rho_{Physics}(\sigma_{dept\_name="Physics"}(instructor))$

ID	name	dept_name	salary
22222	**Einstein**	**Physics**	**95000**
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	**Gold**	**Physics**	**87000**
76543	Singh	Finance	80000

The *instructor* relation

Result

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

$\sigma_{dept\_name="Physics"}(instructor)$

The *Physics* relation

Rename Operation ρ

- The rename operation ρ assigns a new name to a relation or its attributes
- $\rho_{x(A_1,A_2,\ldots,A_n)}(E)$
  - Renames the result of expression E as relation x with attributes $A_1, A_2, \ldots, A_n$
  - Example: $\rho_{Physics(p.id,p.name,p.dept,p.salary)}(\sigma_{dept\_name="Physics"}(instructor))$

ID	name	dept_name	salary
22222	**Einstein**	**Physics**	**95000**
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
983456	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	**Gold**	**Physics**	**87000**
76543	Singh	Finance	80000

The *instructor* relation

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

$\sigma_{dept\_name="Physics"}(instructor)$

Result

p.id	p.name	p.dept	p.salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

The *Physics* relation

Grouping and Aggregation γ

- Tuples with the same attribute values (for a specified list of attributes) are grouped.
- An aggregate function is applied to each group (computing one value for each group).

- Typical aggregate functions
  - *count*
    - count(*): counts all tuples in the group
    - count(A): counts tuples where attribute A is not NULL
  - *sum*
    - sum(A): returns the sum of all non-NULL values of attribute A in the group.
  - *min, max, avg*
    - min(A), max(A), avg(A): returns the minimum, maximum, average of attribute A in a group

- Notation: $\gamma_{L;F}(r)$
  - r: a relation
  - L: list of attributes for grouping
  - F: aggregate function

Grouping and Aggregation γ

- Example
  - Determine the number of students per semester
  - $\gamma_{semester;count(*)}(student)$

name	semester	age
Alice	7	24
Anna	3	20
Bob	3	19
Cathy	5	22
Chris	5	20
David	1	18

$student$

name	semester	age
Alice	7	24
Anna	3	20
Bob	3	19
Cathy	5	22
Chris	5	20
David	1	18

$student$

Result

semester	count(*)
7	1
3	2
5	2
1	1

$\gamma_{semester;count(*)}(student)$

Since $student$ has no NULL values, all four queries below return the same result:

- $\gamma_{semester;count(*)}(student)$
- $\gamma_{semester;count(name)}(student)$
- $\gamma_{semester;count(semester)}(student)$
- $\gamma_{semester;count(age)}(student)$

Grouping and Aggregation γ

- Example
  - Determine the number of students per semester and the minimum age in each group
  - $\gamma_{semester;count(*),\min(age)}(student)$

name	semester	age
Alice	7	24
Anna	3	20
Bob	3	19
Cathy	5	22
Chris	5	20
David	1	18

$student$

name	semester	age
Alice	7	24
Anna	3	20
Bob	3	19
Cathy	5	22
Chris	5	20
David	1	18

$student$

Result

semester	count(*)	min(age)
7	1	24
3	2	19
5	2	20
1	1	18

$\gamma_{semester;count(*),\min(age)}(student)$

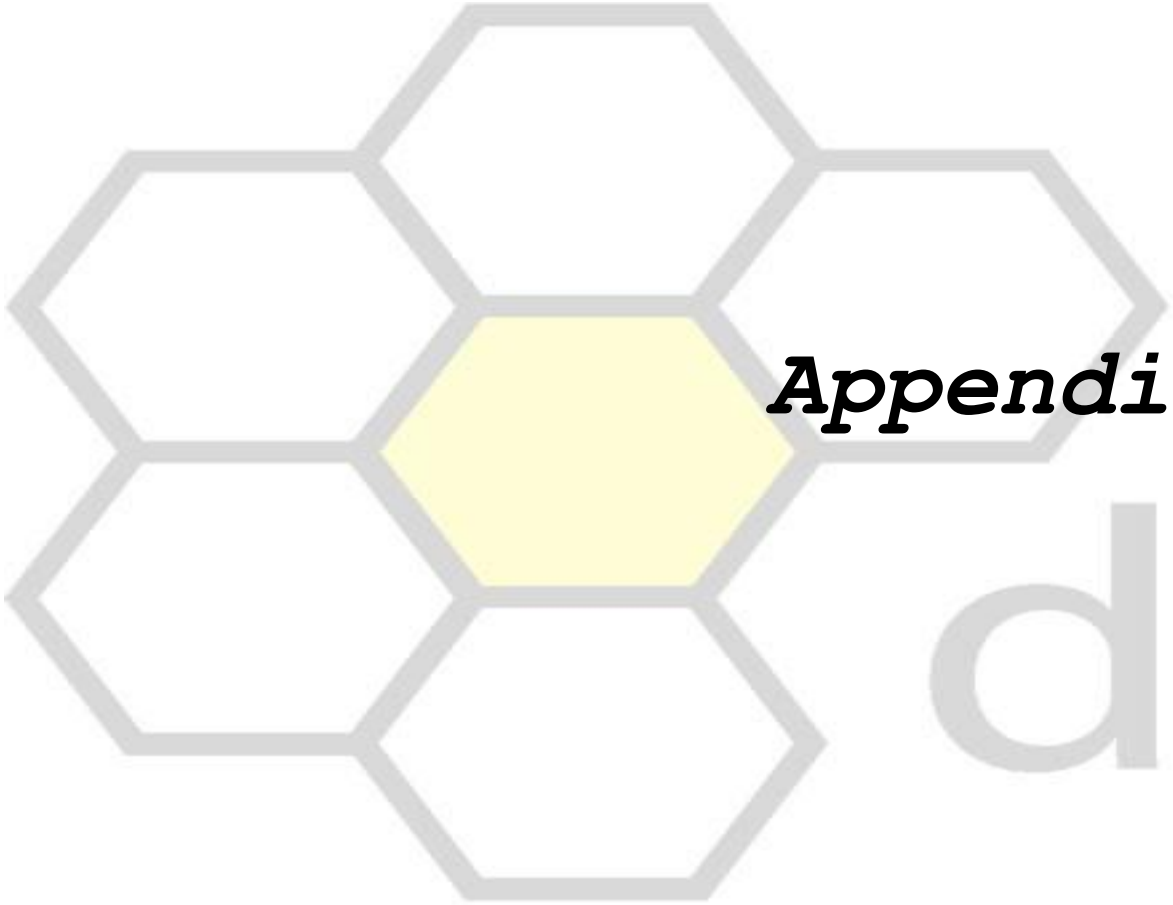
Summary

- Steps of Database Design
- The Relational Model
  - Structure of relational databases
  - Database schema
- Relational Algebra
  - Select
  - Project
  - Cartesian product
  - Join (derived, a combination of select and Cartesian product)
  - Union
  - Difference
  - Intersection (derived, a combination of multiple differences)
  - Rename
  - Group (derived, partitions tuples and applies aggregation)

Reference

- Abraham et al.. Database System Concepts (seventh edition)
- Tiantian Liu. Database Systems(SW5) - 2. Relational Model
- Katja Hose. Database Systems - Relational model and relational algebra.

AALBORG UNIVERSITY



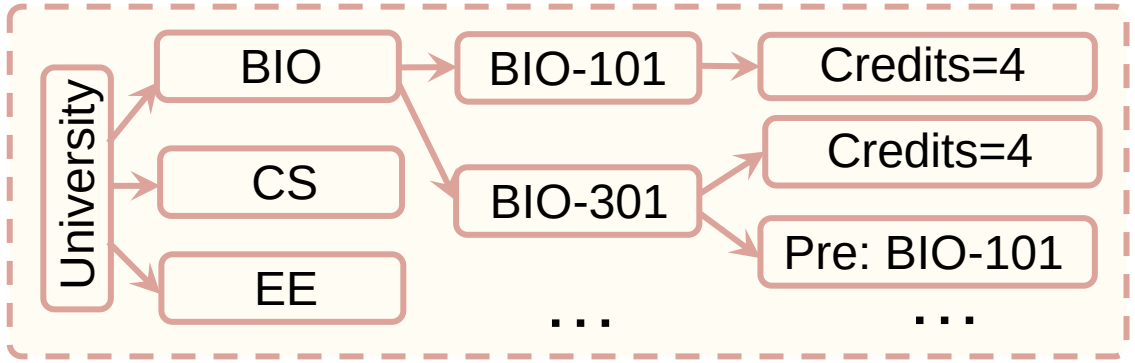
Appendix

History of Relational Model

- A data model is a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.
- Before relational model, most database systems were based on hierarchical and network models.

Raw description

- ***BIO-101 –Biology department. 4 credits. No prerequisites.***
- ***BIO-301 –Biology department. 4 credits. Prerequisite: BIO-101.***
- *CS-101 –Computer Science department. 4 credits. No prerequisites.*
- *CS-190 –Computer Science department. 3 credits. Prerequisite: CS-101.*
- *CS-315 –Computer Science department. 3 credits. Prerequisite: CS-101.*
- *EE-181 –Electrical Engineering department. 3 credits. Prerequisite: PHY-101.*



Hierarchical model

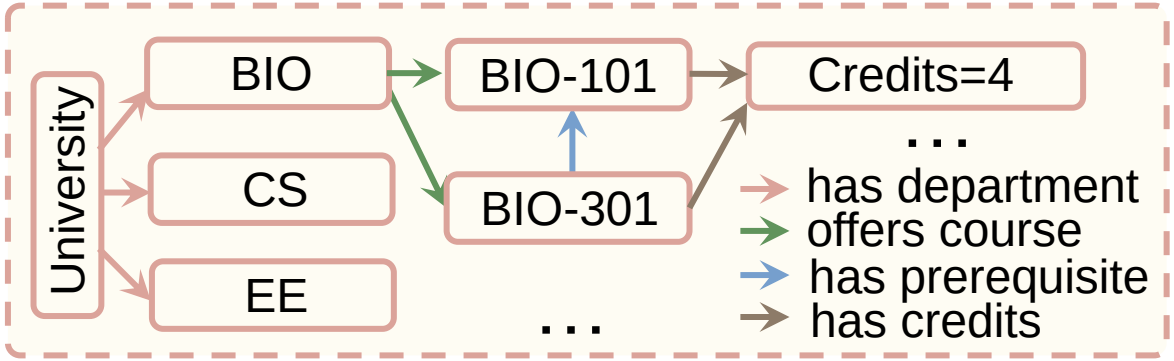
- The hierarchical model arranges records in hierarchy like an organizational chart.
- A node represents a particular entity.
- Difficult to modify.
- Cannot represent all the relation between data.

History of Relational Model

- A data model is a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.
- Before relational model, most database systems were based on hierarchical and network models.

Raw description

- ***BIO-101 –Biology department. 4 credits. No prerequisites.***
- ***BIO-301 –Biology department. 4 credits. Prerequisite: BIO-101.***
- *CS-101 –Computer Science department. 4 credits. No prerequisites.*
- *CS-190 –Computer Science department. 3 credits. Prerequisite: CS-101.*
- *CS-315 –Computer Science department. 3 credits. Prerequisite: CS-101.*
- *EE-181 –Electrical Engineering department. 3 credits. Prerequisite: PHY-101.*



Network model

- A child node can have more than one parent.
- It offers greater flexibility.
- It is slower and harder to maintain.
- Representing a database requires more complicated diagrams.

History of Relational Model

- ✓ The relational model is simpler and more intuitive.
- ✓ It provides greater flexibility and ease of querying.

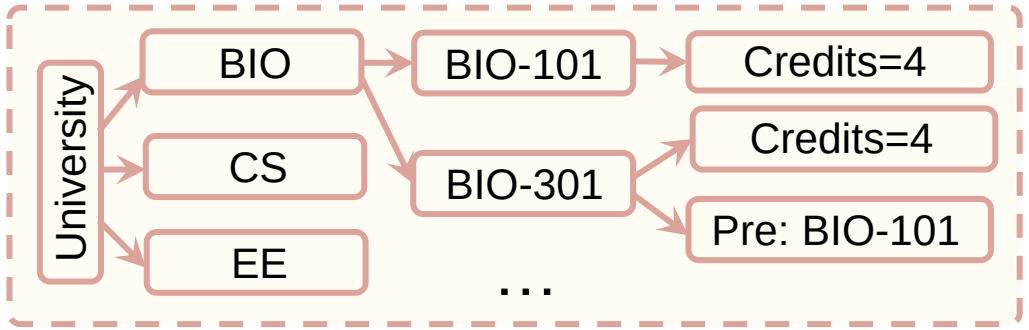
- ***BIO-101 : BIO department. 4 credits. No prerequisites.***
- ***BIO-301 : BIO department. 4 credits. Prerequisite: BIO-101.***
- *CS-101 : CS department. 4 credits. No prerequisites.*
- *CS-190 : CS department. 3 credits. Prerequisite: CS-101.*
- *CS-315 : CS department. 3 credits. Prerequisite: CS-101.*
- *EE-181 : EE department. 3 credits. Prerequisite: PHY-101.*

course

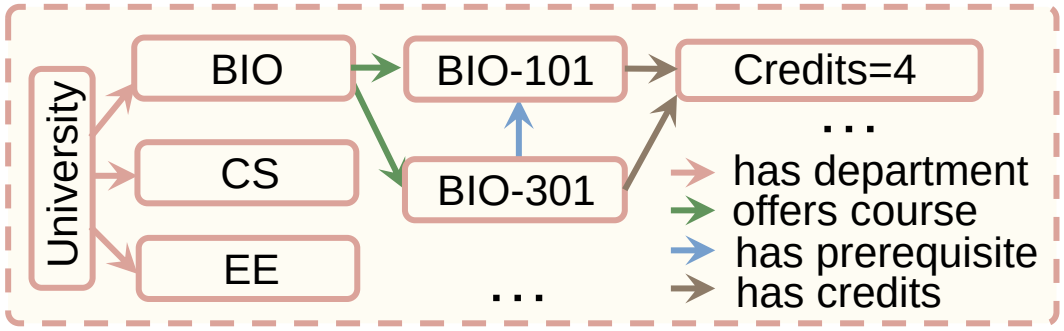
Course_id	dept_name	credits
BIO-101	BIO	4
BIO-301	BIO	4
…	…	…

prereq

Course_id	Prereq_id
BIO-301	BIO-101
…	…



Hierarchical model



Network model

History of Relational Model

- First introduced by Edgar Frank "Ted" Codd (in 1970)
- Uses concept of mathematical relation.
- Codd received the Turing Award in 1983 for this work.
- In the mid-1970s, early RDBMS prototypes were developed by IBM (System R) and UC Berkeley (INteractive Graphics and Retrieval System).
- Current popular RDBMSs:

Oracle

Microsoft Access, SQL Server

PostgreSQL

IBM DB2

Atomic Domain

- Atomic Domain
  - Attribute values are (normally) required to be atomic (indivisible)
  - Example:

id	name	address
001	Elodie	59, SQL Street
002	Felix	103, HTML Road
003	Soren	32, PHP Street

Old table

id	name	street_num	street_name
001	Elodie	59	SQL Street
002	Felix	103	HTML Road
003	Soren	32	PHP Street

✓ Better table

  - When the data is atomic, that means the data has been broken down into the smallest pieces that can't or shouldn't be divided.

This depends on your requirement on getting information.

Being atomic in one scenario may not be so in another.

Atomic Domain

- Atomic Domain
  - Attribute values are (normally) required to be atomic (indivisible)
  - Example:

id	name	address
001	Elodie	59, SQL Street
002	Felix	103, HTML Road
003	Soren	32, PHP Street

  - Domain of address attribute
    - (59, SQL Street), (103, HTML Road), (32, PHP Street)
    - It combines street number + street name
    - Hard to query: e.g., “Find all rows where street = ‘SQL Street’”

More Operations: Assignment Operation ←

- The assignment operation ← stores the result of a relational algebra expression into a temporary relation for later use.
  - Like assignment in a programming language
- Example
  - Find all instructor in the Physics and Music department.

$$\sigma_{dept\_name="Physics"}(instructor) \cup \sigma_{dept\_name="Music"}(instructor)$$

Useful for breaking down steps

$$Physics \leftarrow \sigma_{dept\_name="Physics"}(instructor)$$
$$Music \leftarrow \sigma_{dept\_name="Music"}(instructor)$$
$$Physics \cup Music$$

This makes your solution easier to write and easier for others to understand!

- Note: All attribute names are retained

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example:
  - Find information about courses taught by instructors in the Physics department with salary greater than 90000

  - Query 1
    - $\sigma_{dept\_name="Physicis" \wedge salary>90000}(instructor)$
  - Query 2
    - $\sigma_{dept\_name="Physicis"}(\sigma_{salary>90000}(instructor))$

✓ The two queries are not identical; they are, however, equivalent -- they give the same result on any database.